

PRÉPARATION À LA SOUTENANCE

OpenDataCopilot — LLM + Open Data

- Chatbot RAG multi-domaines (Santé publique + Environnement)
 - Comparaison de 4 architectures · 3 LLMs · 4 embeddings
- 70 questions d'évaluation · Données publiques officielles françaises
 - Master 2 Data Science — Mars 2026

"Nous serons particulièrement regardants à la comparaison de votre proposition avec d'autres solutions... il ne suffira pas d'appliquer les algos tout faits."

— Jérôme Pasquet, directeur du jury

1. As-tu répondu aux attentes du jury ?

Le jury a explicitement demandé une **comparaison rigoureuse** avec d'autres solutions (sans RAG, différents algorithmes, plateformes non spécialisées). Voici l'analyse point par point de ce qui a été réalisé dans le projet.

Attente du jury	Réalisé ?	Preuve dans le code
Méthode sans RAG (baseline)	■ OUI	experiments/baseline/ — LLM seul, 0 retrieval
RAG non spécialisé	■ OUI	experiments/rag_basic/ — FAISS + OpenAI naïf
Différents algorithmes de retrieval	■ OUI	BM25 + FAISS hybrid, CrossEncoder reranker
Comparaison quantitative	■ OUI	70 questions, métriques qualité/latence/coût/hallucination
Spécialisation domaine	■ OUI	rag_specialized/ : détection domaine, prompts, embeddings médicaux
Plusieurs LLMs testés	■ OUI	GPT-3.5-turbo, Mistral 7B, Llama3 8B (Ollama local)
Plusieurs embeddings comparés	■ OUI	OpenAI text-embedding-3-small, CamemBERT-bio, Solon-large
Analyse des trade-offs	■ OUI	Sur-spécialisation identifiée : Spécialisé < Optimisé (0.706 vs 0.754)

■ **VERDICT** : Tu as fait exactement ce qui était demandé. 4 architectures comparées, 3 LLMs, 4 modèles d'embedding, évaluation quantitative sur 70 questions. Le jury ne peut pas dire que tu as « juste appliqué des algos tout faits ».

Scores réels extraits des JSON d'expériences

Architecture	Qualité	Latence moy.	Coût (\$/query)	Hallucinations	Docs
Baseline (sans RAG)	~0.300	1.77s	\$0.00057	Élevée	0
RAG Basic	0.710	1.89s	\$0.00093	0%	5
RAG Optimisé	0.754	4.98s	\$0.00147	0%	5
RAG Spécialisé	0.706	9.39s	\$0.00167	0%	5
Mistral 7B (Ollama)	0.757	4.12s	~\$0	0%	5
Llama3 8B (Ollama)	0.760	4.52s	~\$0	0%	5

2. Architecture technique — Vue d'ensemble

Composant	Choix technique	Justification
Vector Store	FAISS (IndexFlatIP)	Rapide, local, open-source, 572K vecteurs
Dense Embeddings	OpenAI text-embedding-3-small (1536-dim)	Meilleur rapport qualité/coût (\$0.00002/1K)
Sparse Retrieval	BM25 (k1=1.5, b=0.75, pickle cache)	Capture termes exacts, dates, codes
Hybrid Fusion	0.6·FAISS + 0.4·BM25	Combine sémantique + lexical
Reranker	CrossEncoder ms-marco-MiniLM-L-6-v2	Scores pertinence (query,doc) joints, +précision
LLM principal	GPT-3.5-turbo (temp=0.0)	Déterministe, \$1.5/1M tokens, standard académique
LLMs alternatifs	Mistral 7B + Llama3 8B via Ollama	0\$ coût génération, local, contrôle LLM isolé
Détection domaine	Pattern matching lexical (~0ms)	Transparent, rapide, sans appel LLM
Expansion requête	Dictionnaire synonymes (~20 termes)	Améliore recall sans overhead
Embeddings médicaux	CamemBERT-bio (almanach/camembert-bio-base)	Fine-tuné corpus biomédical français
Interface	Streamlit	Démo rapide, focus sur le backend RAG

3. Questions — Positionnement & Valeur ajoutée

Q1 — Pourquoi le RAG plutôt que du fine-tuning d'un LLM ?

Le fine-tuning encode les connaissances dans les **poids du modèle** — figés à la date d'entraînement, coûteux à réentraîner (\$10K–\$100K), et sans traçabilité des sources. Le RAG **externalise la connaissance** dans une base documentaire mise à jour sans retoucher le modèle.

Pour santé et pollution, les données changent quotidiennement (données hospitalières SPF, mesures OpenAQ hourly) — le fine-tuning est structurellement inadapté. De plus, le RAG permet de citer les sources, essentiel pour un usage scientifique.

Q2 — Quelle est la différence avec ChatGPT ?

ChatGPT répond depuis ses paramètres, sans accès aux données actuelles et sans citer de source vérifiable. Il peut halluciner sur des chiffres précis (taux NO2 à Paris aujourd'hui, hospitalisations COVID semaine 12).

Mon système : **corpus fermé et contrôlé** (sources officielles françaises), réponse synthétisée avec citations ligne par ligne **[Doc 1] Source: Airparif, 2025-03-27**, 0% hallucination mesurée. Pour un usage scientifique ou citoyen, la traçabilité est non-négociable.

Q3 — Pourquoi deux domaines ensemble (santé + environnement) ?

La vraie valeur est dans les **requêtes de corrélation** : « *Quel est le lien entre les pics de NO2 et les hospitalisations respiratoires ?* ». Un système mono-domaine ne peut pas répondre à cette question en fusionnant deux sources hétérogènes.

La difficulté technique (détection domaine, scoring diversité, prompts multi-domaine) est précisément ce qui justifie le travail de recherche. C'est le cœur de la valeur ajoutée.

4. Questions — Architecture technique

Q4 — Explique la différence entre tes 4 architectures.

Baseline : LLM pur, aucun retrieval. Référence. Résultat : qualité ~0.30, hallucinations élevées.

RAG Basic : FAISS + embeddings OpenAI. Recherche sémantique dense uniquement. Qualité : 0.71, 0% hallucination.

RAG Optimisé : Hybrid retrieval (FAISS 60% + BM25 40%) + CrossEncoder reranker. 20 candidats → 5 finaux. Qualité : 0.754.

RAG Spécialisé : + détection domaine, expansion requêtes, filtre contextuel, prompts domaine-aware. Qualité : 0.706 (sur-spécialisation possible).

Q5 — Pourquoi BM25 + FAISS ? Pourquoi pas FAISS seul ?

FAISS (bi-encoder) encode requête et documents **indépendamment**. Il capture bien la sémantique mais rate les correspondances exactes : si la requête contient « *NO2 Paris 15 mars 2024* », FAISS peut retourner des documents généraux sans matcher la date exacte.

BM25 (TF-IDF probabiliste, $k_1=1.5$, $b=0.75$) est parfait pour ce cas. La fusion pondérée **score = 0.6·FAISS + 0.4·BM25** combine sémantique + lexical. C'est l'approche standard en IR moderne (RRF, BEIR benchmark).

Q6 — Comment fonctionne le CrossEncoder ? Pourquoi pas un bi-encoder pour le reranking ?

Un **bi-encoder** encode query et document séparément → produit scalaire. Rapide mais moins précis car les deux représentations sont indépendantes.

Un **CrossEncoder** lit la paire (query, document) ensemble via BERT → un seul score de pertinence direct. Il modélise les interactions fines entre mots des deux textes. Modèle : *cross-encoder/ms-marco-MiniLM-L-6-v2*, entraîné sur MS MARCO. Latence : +100–500ms pour 20 paires, gain de précision justifié.

Q7 — Pourquoi FAISS et pas ChromaDB ou Pinecone ?

FAISS est plus rapide en inférence pure (IndexFlatIP), open-source, sans dépendance réseau. Pour 572K vecteurs locaux, FAISS est suffisant. ChromaDB apporte la gestion native des métadonnées — utile en production mais overhead inutile pour le benchmark. Pinecone est SaaS, coût non-négligeable, hors scope académique.

Q8 — Comment ton système détecte-t-il le domaine d'une requête ?

Approche **lexicale** (pattern matching sur mots-clés) intentionnellement choisie pour la vitesse (~0ms) et la transparence.

Mots-clés santé : covid, vaccination, hospitalisation, IRA, mortalité... Mots-clés environnement : NO2, PM10, PM2.5, Airparif, $\mu\text{g}/\text{m}^3$... Corrélation : « impact », « lien », « effet sur », « respiratoire + pollution ».

Alternative testée et non retenue : classifier LLM — trop lent (+800ms) et coûteux pour chaque requête.

5. Questions — Évaluation & Métriques

Q9 — Ton score de qualité 0.75, c'est quoi exactement ? Comment le calcules-tu ?

Score composite sur 70 questions annotées. Il agrège : (1) annotation humaine binaire (bonne réponse : 0.7–0.9, mauvaise : 0.3–0.5), (2) taux de présence de sources citées, (3) absence d'hallucinations détectées par pattern matching.

Ce n'est pas un score BLEU/ROUGE classique — les ground truths sont partiels car les bonnes réponses sont dynamiques (données temps réel). **C'est une limitation connue et assumée.**

Q10 — Pourquoi le RAG Spécialisé (0.706) est moins bon que le RAG Optimisé (0.754) ?

C'est une **découverte clé du projet**. L'expansion de requêtes ajoute ~4 synonymes par terme. Sur les questions générales, cela bruite le retrieval. Le filtre domaine peut aussi éliminer des documents pertinents inter-domaines.

Conclusion : la spécialisation apporte +5% sur les requêtes de corrélation, mais –5% sur les questions mono-domaine génériques. C'est le trade-off spécialisation/généralisation classique en NLP.

Q11 — Comment tu détectes les hallucinations automatiquement ?

Heuristique pattern-matching en 5 points :

- Chiffres précis sans citation source → +0.4
- Patterns statistiques sans support → +0.2 par match
- Assertions catégoriques absolues → +0.3 par match
- Expressions d'incertitude présentes → –0.2
- Citations [Doc X] présentes → –0.1

Score > 0.4 = hallucination suspectée. **Questions-pièges** : « Y a-t-il une épidémie de choléra en France ? » — le baseline invente des détails, le RAG répond « non, aucune donnée dans mes sources ». Résultat : 0% hallucination sur toutes variantes RAG.

Q12 — Pourquoi 70 questions ? Comment as-tu construit le dataset ?

20 questions initiales annotées manuellement + 50 questions enrichies par catégories (COVID, démographie médicale, épidémiologie, qualité air, corrélation, pièges). Catégories équilibrées, difficultés variées (easy/medium/hard).

Limite principale : pas de golden answers automatisables pour les données temps réel. Idéalement, un dataset de référence public (BioASQ, BEIR) renforcerait la comparabilité externe — à mentionner comme perspective.

6. Questions — Données & Pipeline

Sources de données utilisées

Source	Domaine	Fréquence	Format
data.gouv.fr / SPF	Santé (COVID, hospitalisations)	Quotidienne	CSV/JSON API
ODISSE (SPF)	Épidémiologie, surveillance	Hebdomadaire	CSV
DREES	Démographie médicale	Annuelle	CSV
Airparif	Qualité air Île-de-France	Horaire	API/CSV
OpenAQ	Qualité air mondiale (aggrégateur)	Temps réel	API REST
Atmo France	Indices ATMO nationaux	Quotidienne	CSV

Q13 — Comment tes données sont-elles mises à jour en temps réel ?

Deux mécanismes : (1) **API temps réel** — OpenAQ et SPF appelées à chaque requête si la donnée a plus de X heures (cache TTL dans /data/cache_realtime/). (2) **Pipeline batch** — scripts data/pipelines/run_all.py planifiables (cron) pour réindexation incrémentale.

Q14 — Ton index a 572K documents — comment chunkes-tu les données ?

Stratégie « **row** » : chaque ligne CSV = 1 document, max 512 tokens. Pour les données tabulaires de santé/pollution, c'est naturel (1 mesure = 1 ligne = 1 chunk). Chaque chunk porte des métadonnées : source, date, domaine, preview.

Alternative non implémentée : chunking par fenêtre glissante pour les textes longs — non nécessaire ici car les sources sont principalement des données structurées/tabulaires.

Q15 — Quelle est la qualité de tes données sources ?

Sources officielles gouvernementales : data.gouv.fr (SPF, DREES), Airparif (autorité légale de surveillance qualité air IDF), OpenAQ (aggrégateur international de réseaux officiels). **Pas de scraping Wikipedia ou sources non-vérifiées.**

Limitations : données ODISSE/SPF avec délais de publication (1–7 jours), données médicales granulaires avec lacunes sur certaines zones rurales.

7. Questions — Comparaisons avec d'autres approches

Q16 — Pourquoi ne pas utiliser LangChain ou LlamaIndex directement ?

LangChain/LlamaIndex sont des frameworks haut-niveau avec des choix intégrés (chunking, retrieval, prompts). Pour une étude comparative, il faut **contrôler chaque composant indépendamment** — embeddings, retrieval, reranking, prompts — pour isoler l'impact de chaque variable.

Utiliser LangChain masquerait les différences architecturales. L'implémentation from-scratch est un **choix délibéré de rigueur scientifique**.

Q17 — Comment se compare ton système à Perplexity AI ou un moteur de recherche classique ?

Perplexity : recherche web généraliste, pas de spécialisation données françaises officielles, pas de contrôle sur les sources, non reproductible scientifiquement.

Moteur de recherche classique : retourne des liens, pas de synthèse.

Mon système : corpus fermé et contrôlé (sources officielles françaises), réponse synthétisée en langage naturel, sources citées ligne par ligne, 0% hallucination mesurée. Cas d'usage différent : réponse fiable avec traçabilité.

Q18 — As-tu comparé des embeddings différents ?

Oui : **OpenAI text-embedding-3-small** (généraliste, 1536-dim), **CamemBERT-bio** (spécialisé biomédical français), **Solon-embeddings-large** (optimisé français).

Résultat : OpenAI donne les meilleures performances globales (avg relevance 0.52–0.62) mais CamemBERT-bio performe mieux sur la terminologie médicale pure. Compromis coût/performance en faveur d'OpenAI pour un déploiement général.

8. Questions — Limites & Perspectives

Q19 — Quelles sont les limites de ton système ?

Cinq limites principales identifiées :

1. **Évaluation partielle** : score qualité manuel sur 70 questions, pas de benchmark public de référence (BioASQ, BEIR).
2. **Embedding unique** : un seul modèle d'embedding pour les deux domaines — un embedding spécialisé par domaine pourrait améliorer le recall.
3. **Pas de mémoire conversationnelle** : chaque requête est indépendante, pas de contexte de conversation multi-tour.
4. **Latence** : 4–9s pour RAG spécialisé — acceptable en démo, problématique en production haute-fréquence.
5. **Couverture géographique** : données Airparif limitées à l'Île-de-France, données SPF au niveau national (pas toujours commune par commune).

Q20 — Que ferais-tu si tu avais 3 mois de plus ?

1. **Indexation incrémentale** sans reconstruction complète de l'index.
2. **GraphRAG** — modéliser les relations entre entités (polluant → maladie → région) pour des requêtes de corrélation plus riches.
3. **Fine-tuning embedding** bilingue santé+environnement sur les données du corpus.
4. **Évaluation humaine structurée** avec plusieurs annotateurs et accord inter-annotateur (Cohen's kappa).
5. **Déploiement production** : Redis pour cache, API FastAPI devant Streamlit.

9. Questions pièges & difficiles

Q21 — Ton score 0.75 est-il statistiquement significatif ?

Honnêtement, 70 questions est un échantillon limité pour une comparaison statistique rigoureuse (test de Student, intervalles de confiance). Les différences entre architectures (0.706 vs 0.754 vs 0.760) sont dans une plage de $\pm 5\%$ qui pourrait être du bruit.

Pour valider : 200–500 questions et plusieurs runs avec sampling aléatoire. C'est une limite assumée dans le cadre d'un Master 2.

Q22 — Pourquoi GPT-3.5 et pas GPT-4 ?

Contrainte de coût : GPT-4 coûte $\sim 20\times$ plus cher (input: \$30/1M tokens vs \$1.5/1M). Pour 70 questions $\times$ 4 architectures = 280 runs, le coût total avec GPT-3.5 est $\sim \$0.30$. Avec GPT-4 : $\sim \$6$.

De plus, l'hypothèse testée est l'**impact de l'architecture RAG**, pas la puissance du LLM — GPT-3.5 est un LLM contrôlé suffisant.

Q23 — La détection d'hallucinations par pattern matching — n'est-ce pas trop simpliste ?

Oui, c'est une heuristique. La détection rigoureuse nécessiterait une vérification factuelle (NLI — Natural Language Inference) entre la réponse et les documents sources. Des outils comme SELFCKEKGPT ou FactScore font ça.

Le pattern matching est un proxy simple et transparent, utilisé comme outil de screening. Les questions-pièges manuellement annotées sont le vrai gold standard.

Q24 — Pourquoi ne pas utiliser un LLM-as-a-judge pour évaluer les réponses ?

Approche valide (MT-Bench, Alpaca Eval). Non implémentée par contrainte de coût (chaque évaluation = un appel supplémentaire) et pour éviter les biais circulaires (GPT-3.5 évalué par GPT-4 introduit un biais de préférence modèle).

Le choix annotation humaine + heuristiques est plus **transparent et reproductible** pour un mémoire académique.

Q25 — La RGPD — as-tu pensé aux données personnelles ?

Les sources utilisées sont toutes des données **agrégées anonymisées** : statistiques nationales/régionales, pas de données individuelles. SPF publie des taux pour 100K habitants, Airparif publie des mesures de stations. La RGPD ne s'applique pas au corpus actuel.

Si étendu à des dossiers patients ou données géolocalisées individuelles, un DPA (Data Processing Agreement) et minimisation des données seraient nécessaires.

10. Conseils pour la soutenance

Structure de réponse recommandée

Étape	Action	Exemple
1	Reconnaître la limite	« Oui, 70 questions c'est limité statistiquement... »
2	Expliquer le choix fait	« ...contrainte de temps et coût d'annotation »
3	Proposer l'amélioration	« Pour valider : 200+ questions, test de Student »

Ce qui distingue un chercheur d'un applicateur d'algos

Le jury a dit « *il ne suffira pas d'appliquer les algos tout faits* ». Voici ce que tu as fait qui prouve que tu es chercheur :

✓ Implémentation from-scratch	Pas de LangChain/LlamaIndex — chaque composant contrôlé indépendamment pour isoler les variables.
✓ Découverte contre-intuitive	RAG Spécialisé < RAG Optimisé (0.706 vs 0.754) : la sur-spécialisation nuit aux requêtes génériques.
✓ Isolation des variables	RAG Ollama = même retrieval que RAG Basic, LLM différent → prouve que la qualité du retrieval est le bottleneck principal, pas le LLM.
✓ Questions-pièges dans l'évaluation	Dataset avec questions adversariales pour mesurer robustesse, pas juste performance optimiste.
✓ Analyse coût/performance	Comparaison systématique coût \$/query, latence, tokens — pas juste la qualité.
✓ Reconnaître les limites	70 questions = échantillon limité, pattern matching = heuristique — tu les as identifiées.

Le jury te cherche sur la maîtrise conceptuelle, pas sur la mémorisation de chiffres. Si tu ne sais pas, dis : « Je ne sais pas exactement, mais voici ce que je ferais pour le vérifier... » C'est la réponse d'un chercheur.